Receipt & Invoice Digitizer

AI-Powered Bills Management System

Team- D

Milestone- II

HTML • CSS • JavaScript • Python • SQL • OCR

Team- D members

Trishala, Sriya, Rithwik, Mastan and Atul

HTML • CSS • JavaScript • Python • SQL • OCR

Project Overview

Purpose

A comprehensive web application designed to digitize and manage receipts and invoices efficiently, eliminating the hassle of paper-based record keeping.

Key Benefits

Automated data extraction using OCR technology

Secure cloud storage for all documents

Easy search and retrieval of financial records

Analytics and reporting capabilities

Core Features

User Authentication

Secure registration and login system with password encryption

Document Upload

Support for images and PDFs with drag-and-drop functionality

OCR Processing

Automatic extraction of vendor, date, amount, and tax information

Smart Search

Filter receipts by vendor, date, and invoice number

Data Management

View, edit, and delete receipts with ease

Analytics Dashboard

Visualize spending patterns and generate reports

Technology Stack

Backend

Flask (Python Web Framework)
SQLite Database
bcrypt (Password Hashing)
Werkzeug (File Handling)
Session-based Authentication

OCR Integration (Planned)

Tesseract OCR / Google Vision API
Automatic data extraction

Frontend

HTML5, CSS3, JavaScript
Responsive Design
Modern UI with gradient theme
Drag-and-drop file upload
AJAX for dynamic updates

Design Features

Custom pink gradient theme
Smooth animations
Clean, modern interface

Frontend Architecture

Code Organisation

Clean, modular code structure with separation of concerns:

HTML Structure

- Semantic HTML5 elements
- Navbar with navigation items
- Centered header with large title
- Card-based layout
- Table for receipts display

JavaScript Logic

- JWT authentication check
- File upload handling
- Drag & drop implementation
- API integration (fetch)
- Dynamic table rendering

CSS Styling

- Custom pink gradient background
- Flexbox for layouts
- Hover effects & transitions
- Responsive media queries
- Custom scrollbar styling

Design

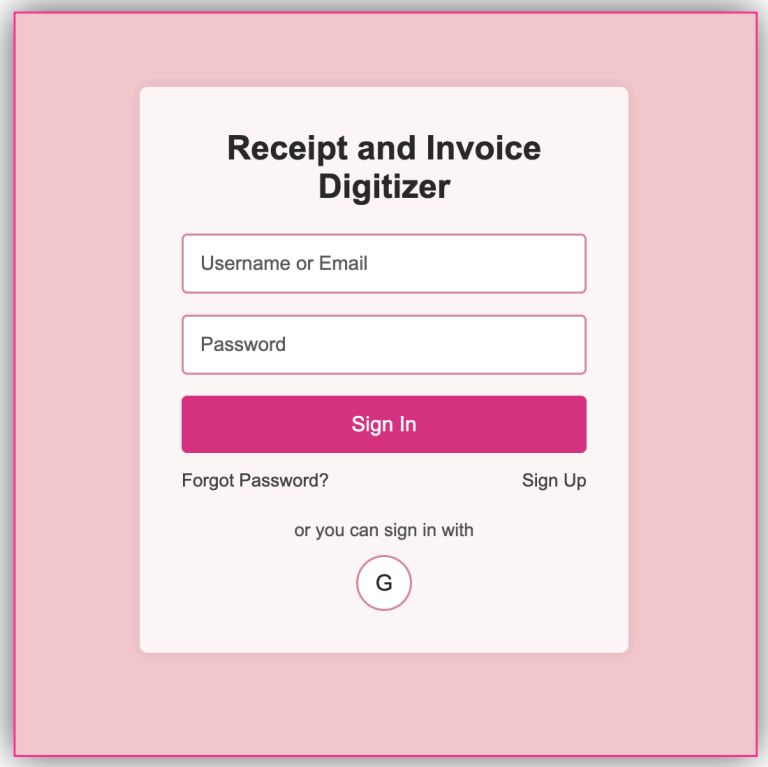
- Custom fonts: DM Sans & Space Grotesk
- Gradient backgrounds: #f5d5d8 → #e8b4b8 → #d4989c
- Rounded corners and glassmorphism effects

Frontend Structure

Login and Forgot Password

Login Design

1. Username or Email field for user identification
2. Password field for secure authentication
3. Sign In button to access the application
4. Forgot Password option for credential recovery
5. Sign Up option for new user registration.

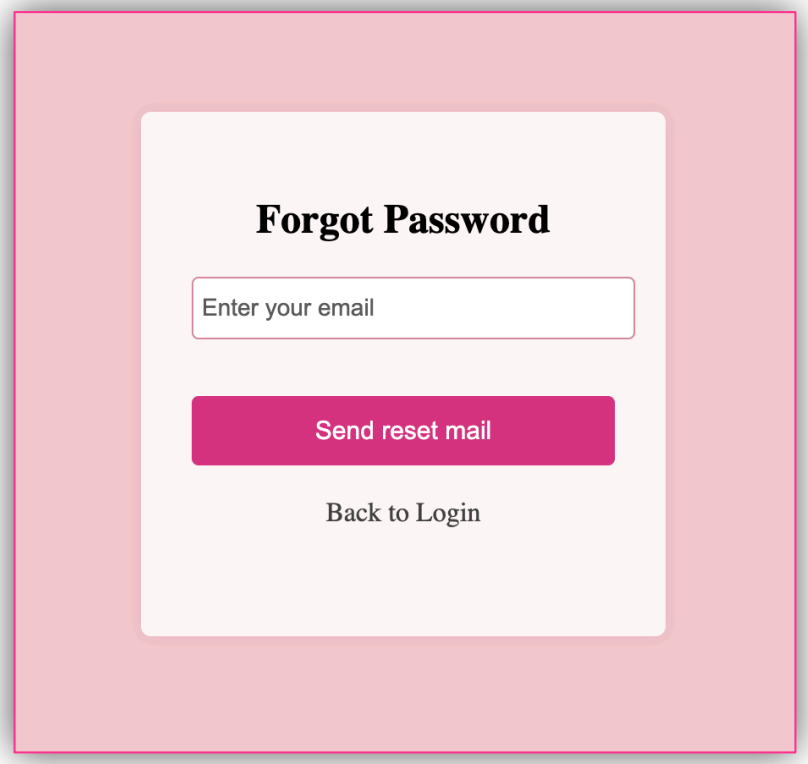


Frontend Structure

Login and Forgot Password

Forgot password Design

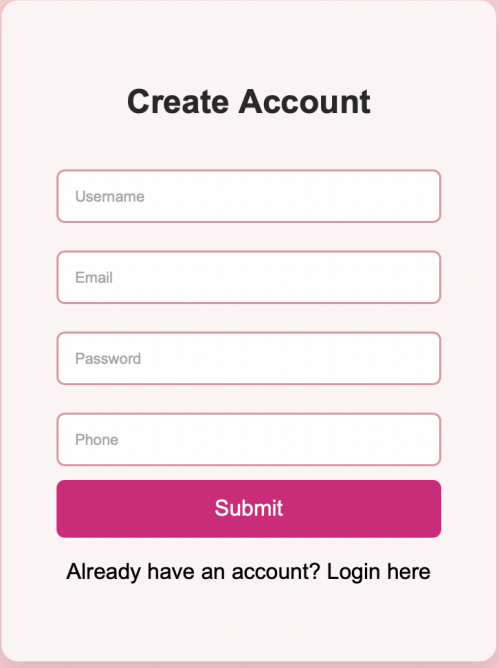
1. The forgot password page has an email input field for verification.
2. It provides an option to send a password reset email.
3. A back-to-login option is included for easy navigation.



Frontend Structure

Registration Page

1. The Registration Page provides a clean and user-friendly interface for new users to create an account.
2. The design follows a modern card-based layout with a clear call-to-action button.
3. The form includes proper validation for all input fields, ensuring data integrity and providing real-time feedback to users.
4. The interface offers a seamless transition to login for existing users, reducing friction in the user authentication flow.



Frontend Structure

Dashboard

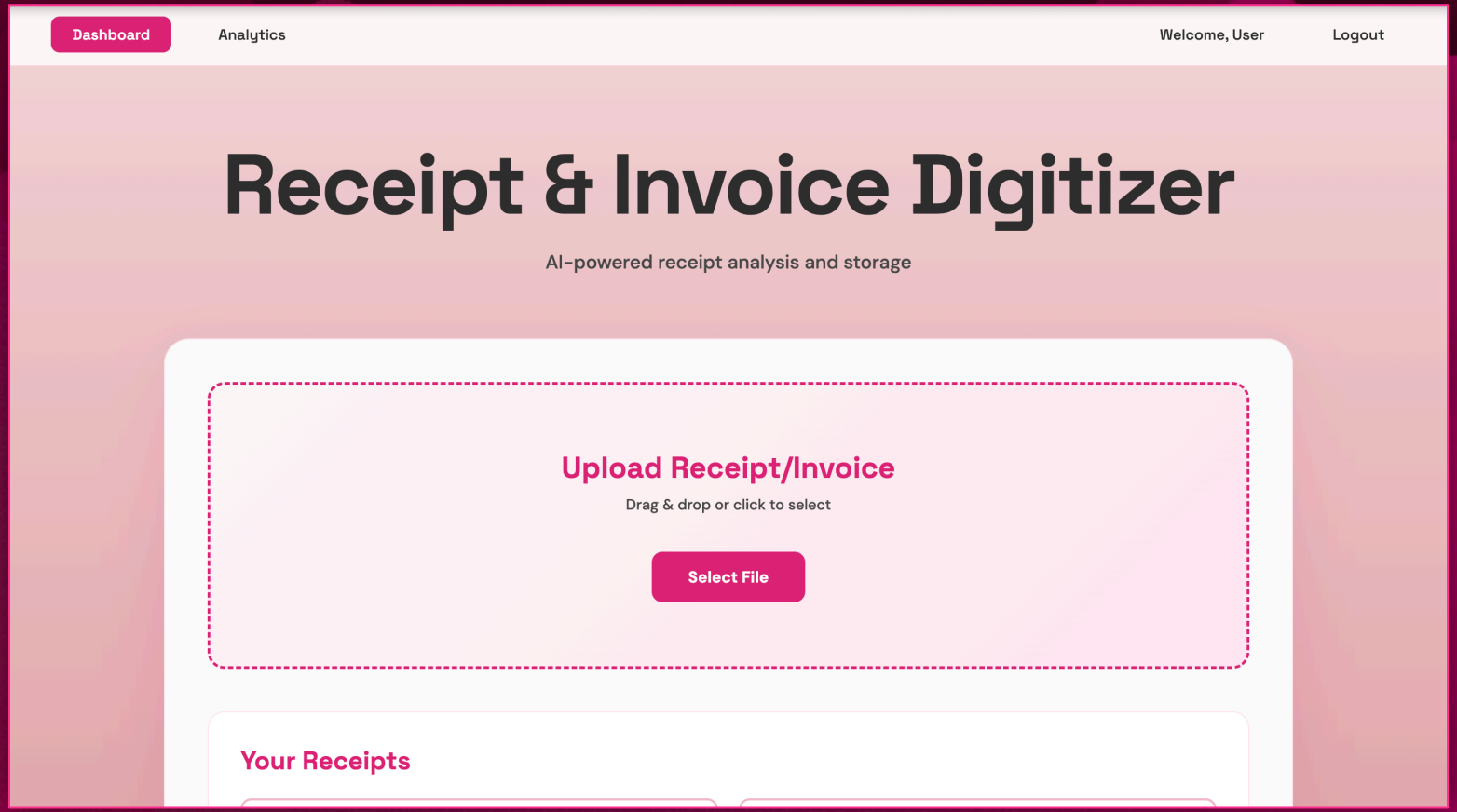
1. **Sticky Navbar**
   Automatically hides on scroll down, reappears on scroll up for better UX

2. **Centered Hero Section**
   Large 80px title with gradient background creates strong visual impact

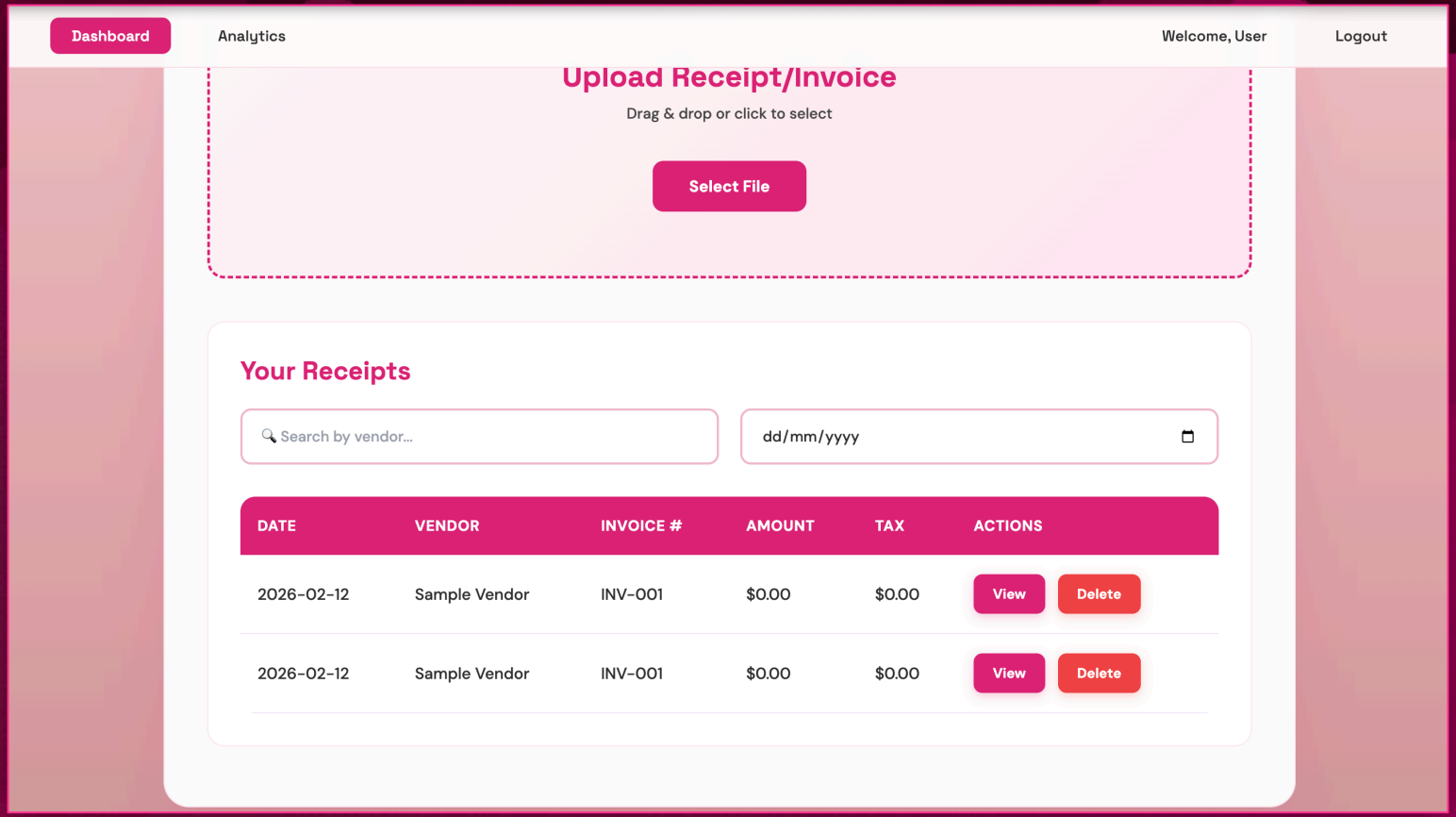
3. **Upload Zone**
   Dashed border, hover effects, and drag-drop functionality for easy file uploads

4. **Data Table**
   Clean table design with search filters and action buttons for each receipt

Frontend Structure



Frontend Structure



Technology Stack

Backend

Flask (Python Web Framework)
SQLite Database
bcrypt (Password Hashing)
Werkzeug (File Handling)
Session-based Authentication

OCR Integration (Planned)

Tesseract OCR / Google Vision API
Automatic data extraction

Frontend

HTML5, CSS3, JavaScript
Responsive Design
Modern UI with gradient theme
Drag-and-drop file upload
AJAX for dynamic updates

Design Features

Custom pink gradient theme
Smooth animations
Clean, modern interface

Backend Architecture

Python & Flask

Core development framework for building scalable RESTful services.

SQLite

Lightweight relational database for structured user information storage.

bcrypt

Industry-standard password hashing to ensure cryptographic security.

Backend Development

- Designed core Flask backend architecture
- Developed secure Login/Register endpoints
- Integrated backend logic with HTML forms
- Implemented bcrypt password hashing
- Configured and optimized SQLite storage
- Resolved complex integration bugs

Authentication Flow

User Input
Form submission via Frontend

API Processing
Endpoint routing in Flask

Hashing
Secure bcrypt encryption

DB Storage
SQLite Commit & Validate

Verification
Session-based response

Database Schema

Users Table

id (Primary Key)
username (Unique)
email (Unique)
password (Hashed)
phone

Database Features

SQLite database with automatic initialization
Foreign key relationships ensure data integrity
Automatic timestamp tracking for uploads
Efficient indexing on username and email fields

Receipts Table

receipt_id (Primary Key)
user_id (Foreign Key)
filename
upload_date
vendor_name, invoice_number
date, total_amount, tax

Security & Data Protection

🔒 Password Security
bcrypt hashing with unique salts for each password

🛡️ Session Management
Secure session tokens with automatic expiration

📁 File Validation
Strict file type and size checks before upload

🔐 Access Control
User-specific data isolation and authentication

🚫 SQL Injection Prevention
Parameterized queries for all database operations

✅ Input Sanitization
Werkzeug secure filename generation

OCR Integration

Automated OCR + Local LLM Structuring Architecture

1

Drag & Drop Upload

User uploads receipt image or PDF

2

Flask Upload API

OCRfile via Flask route

3

Tesseract OCR

Text extraction from receipt image

4

Gemma 2B (Local LLM)

LLM structures extracted text into JSON

5

Regex Fallback (Safety Layer)

Regex validates critical fields

6

SQLite Database + Dashboard

Saved in SQLite
Displayed in Dashboard

User Workflow

1. Register / Login
2. Upload Receipt
3. OCR Processing
4. Review & Edit
5. Search & Filter
6. View Analytics

Future Enhancements

Google Sign in- Authentication

Google authentication lets users log in using their Google account instead of creating a new username/password.

Export Functionality

Export receipts to Excel, CSV, or PDF reports

View Analytics

Visual summary of user activity

UI/UX

For a dynamic, scalable, and maintainable user interfaces.

Multi-language Support

Support for receipts in different languages

Chat Bot

An AI assistant inside dashboard to help users interact with their receipts

Developments

Google Sign in- Authentication

Google authentication lets users log in using their Google account instead of creating a new username/password.

OCR development

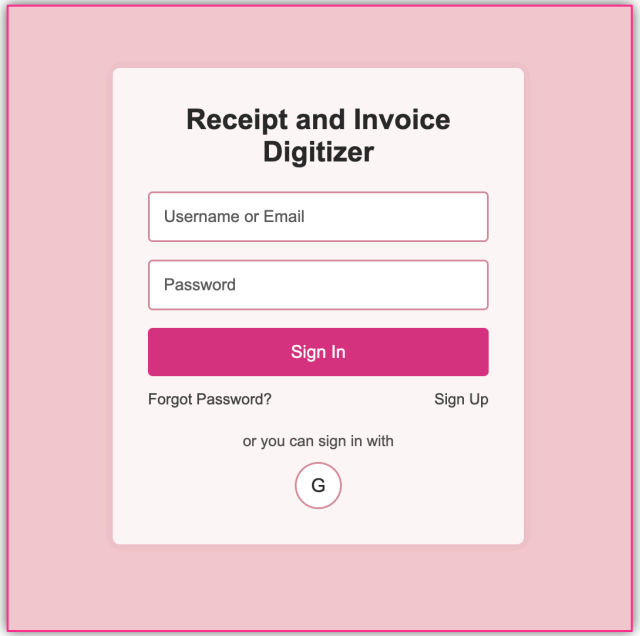
An advancement in the ocr model

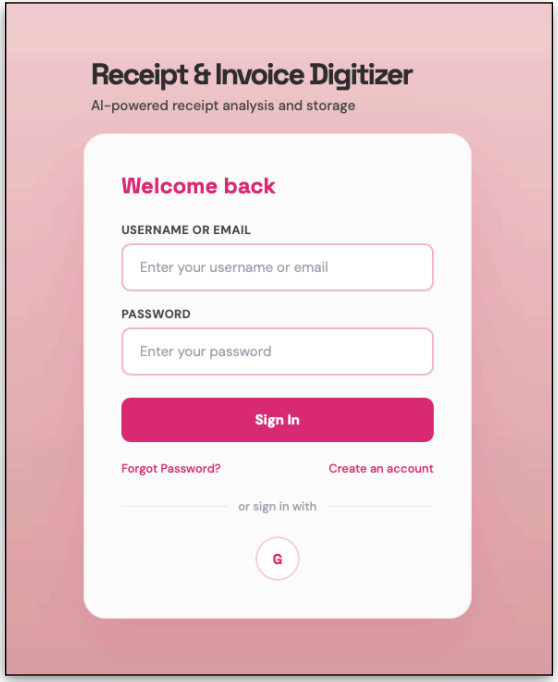
UI/UX

The older UI had been changed to match the theme

Frontend Structure

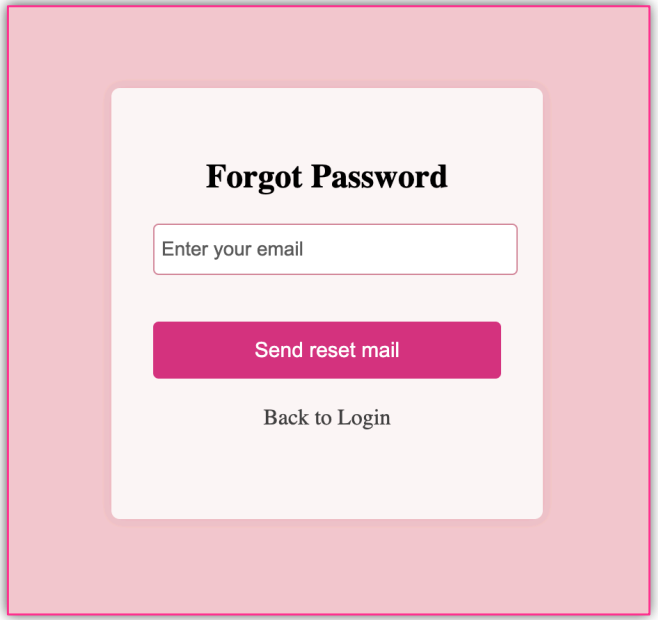
Login page

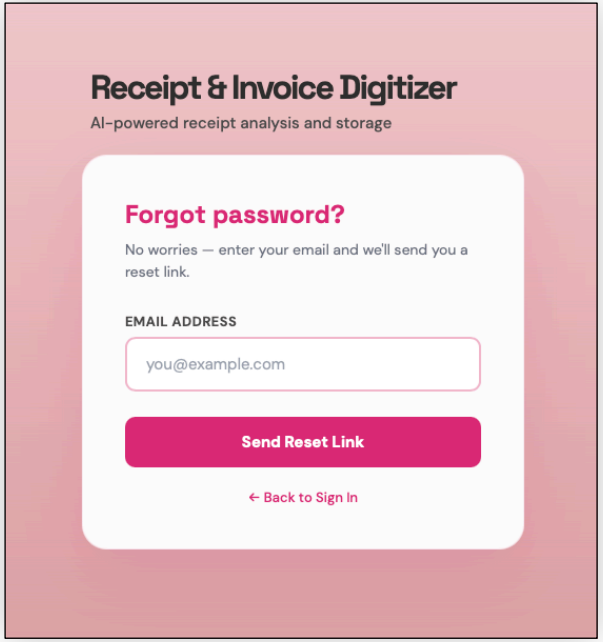




Frontend Structure

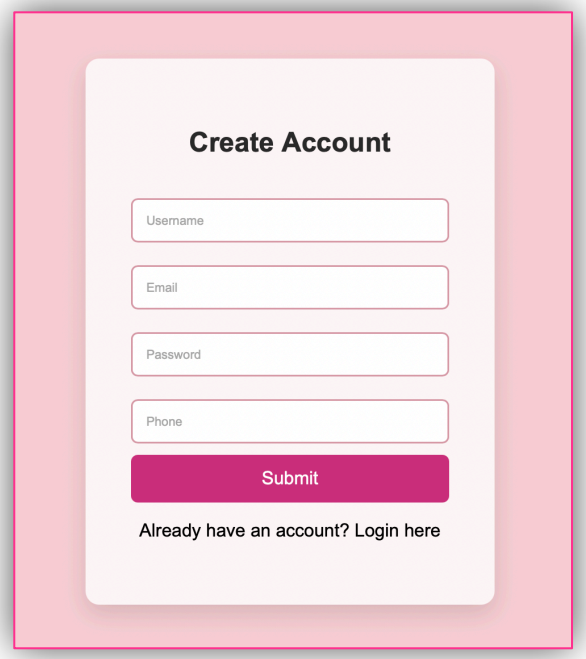
Forgot Password

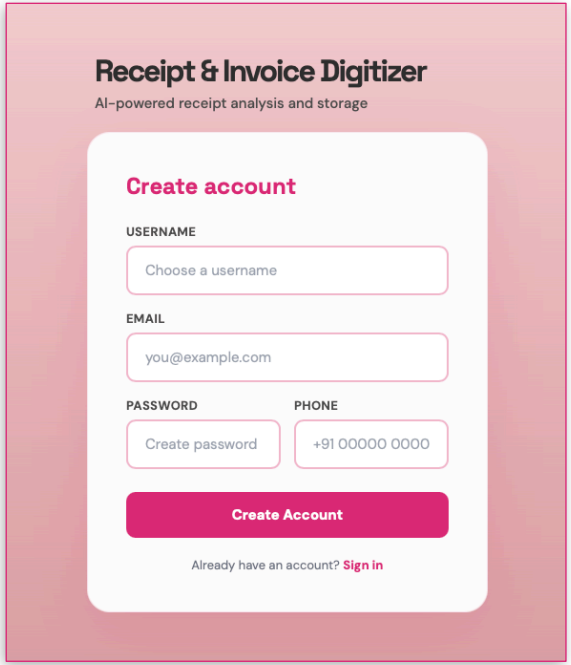




Frontend Structure

Registration Page





Evolution of the OCR Pipeline

Phase 1: PyTesseract

- Poor accuracy on receipts
- Failed to recognise decimal values
- Broken text formatting

Phase 2: PyTesseract + Gemma LLM

- Structured text into JSON
- Hallucinated fields & inconsistent output

Phase 3: PyTesseract + Gemma + Regex

- Regex validation for totals & dates
- Not scalable for varying formats

Final: PaddleOCR + Mistral LLM

- Improved accuracy & decimal detection
- Cleaner JSON output
- No regex dependency

Evolution of the OCR Pipeline

PyTesseract vs PaddleOCR Comparison

PyTesseract

- Inconsistent decimal recognition
- Poor noise handling
- Broken text alignment

PaddleOCR

- Accurate decimal recognition
- Better layout preservation
- High accuracy on receipts
- Better multi-font handling

Evolution of the OCR Pipeline

LLM Upgrade: Gemma → Mistral

Problems with Gemma

- Inconsistent JSON formatting
- Hallucinated structured values
- Required regex fallback layer

Why Mistral?

- Better instruction following
- More stable structured output
- Reduced hallucination
- Improved field extraction accuracy

Final OCR + LLM Architecture

1. User Upload (Image/PDF)
2. Flask API Route
3. PaddleOCR Text Extraction
4. Mistral LLM Structuring to JSON
5. Store in SQLite Database
6. Display in Analytics Dashboard

Thank You!

AI-Powered Bills Management System

Team- D

Presented By: Trishala, Sriya, Rithwik, Mastan and Atul